Project Documentation

Automated Network Request Management

A ServiceNow scoped low-code application built with App Engine Studio

30 October 2026

Note on this documentation format
The supplied format was written for MERN-stack projects (React, Node.js, Express.js, MongoDB). This project is not a MERN application: it is a ServiceNow scoped application that uses native platform features. Each section below is answered with the ServiceNow equivalent, and items that do not apply (npm commands, REST endpoints, JWT tokens and so on) are stated as not applicable instead of being invented.

1. Introduction

- **Project Title:** Automated Network Request Management
- **Team ID:** (not provided)
- **Team Members:** names and roles were not provided; please fill in the table below.

Name	Role

2. Project Overview

Purpose

Organizations receive network service requests (new connections, existing connections and related services). Without a structured process these requests suffer from incomplete information, repeated manual entry, difficult tracking, manual approval delays, unclear responsibility, missing notifications and inconsistent validation. The project organizes the request lifecycle through a ServiceNow Service Catalog item and Flow Designer flows. It is a prototype in a developer instance, not a production network provisioning platform.

Features

- Network Request catalog item in the Service Portal (/sp) with seven main variables and required-field settings.
- Requester Information variable set: User Name and Email update when the selected user changes; Phone Number fills when the user's profile has a mobile phone number.
- Catalog UI Policy 'Show Existing ID for Existing Connection': Existing ID is visible and mandatory only for Existing connections.
- Network Request flow: stores the request in a custom Database Tables table, sends a 'Request has been Created' email and asks for approval.
- Approved or Rejected outcomes update Approval Status and Assignment to; approval records are shown in a related list on the request record.
- Network Task Table to track work with a separate Work Status (task records are currently created manually).
- Network Task Completion Email flow: sends 'Request has been Resolved' when Work Status changes to Completed.

3. Architecture

ServiceNow replaces the three MERN tiers as follows: the Service Portal and catalog form act as the frontend, Flow Designer acts as the backend logic, and platform-managed custom tables act as the database. See the Solution Architecture document for the diagram.

Frontend (Service Portal and Service Catalog)

Requesters open the Service Portal at `/sp` and the **Network Request** catalog item. No custom React, Angular or other framework is used. The form variables are:

Question	Internal name	Type / behaviour
Requested For	requested_for	Single Line Text; required
Mobile Number	mobile_number	Single Line Text; not marked mandatory on the recorded form (no custom ten-digit script)
Type of Connection	type_of_connection	Multiple Choice: New (new, default), Existing (existing); required
Enter your Existing ID	enter_your_existing_id	Single Line Text; conditionally visible and mandatory through the Catalog UI Policy
Total Amount	total_amount	Single Line Text; required
Mode of Payment	mode_of_payment	Multiple Choice: UPI (default), Card; required
Address	address	Multi Line Text; required

The **Requester Information** variable set contains Opened on behalf of (user reference), Email, User Name and Phone Number. The Catalog UI Policy applies when `type_of_connection` is Existing (on load enabled, reverse if false enabled).

Backend (Flow Designer)

Flow: Network Request (activated; trigger: Catalog Item Requested)

#	Action	Configuration
1	Get Catalog Variables	requested_for, mobile_number, type_of_connection, total_amount, mode_of_payment, address
2	Create Record	Creates a Database Tables record: address, mobile number, mode of payment, requested for, total amount and type of connection from the variables; Database Number = requested item number; Approval Status = Requested
3	Send Email	Subject 'Request has been Created'; request number and 'resolved within 2 Business working days'; signed Network Team
4	Ask For Approval	Reason 'Waiting for approval'; approver in testing: System Administrator
5	If / Else Update Record	Approved: Assignment to = ITIL User, Approval Status = Approved. Else: Assignment to = Abraham Lincoln, Approval Status = Rejected

Flow: Network Task Completion Email: trigger Record Updated on Network Task Table when Work Status changes to Completed (run for each unique change); Send Email to Requested For's email with subject 'Request has been Resolved' and the requested-for name in the greeting. A separate Email Notification 'Network Task Resolved' also exists, but the demonstrated resolution email came from this flow.
Approval records are shown on the Database Tables record through a related list that filters `sysapproval_approver` by the parent record's table and sys_id (a related-list filter, not a custom approval engine).

Database (ServiceNow custom tables)

Data is stored in the platform-managed ServiceNow database. No MongoDB, MySQL or other external database is used.

Table and field	Purpose
Database Tables **x_2107483_automa_0_database_tables**	Stores request data and approval state. Fields: Database Number, Assignment to, Mobile Number, Total Amount, Mode of Payment, Type of Connection, Customer Address, Approval Status, Requested For.

Table and field	Purpose
Network Task Table x_2107483_automa_0_network_task_table	Tracks work. Fields: Number (auto-generated), Database Number (reference), Request Number (reference), Requested For (reference), Approval Status, Work Status, Assigned to (reference), Description, Work Notes.

Approval Status (approved or rejected) and Work Status (new, in progress or completed) are different concepts and are stored separately.

4. Setup Instructions

Prerequisites

- A ServiceNow Personal Developer Instance (PDI) and an administrator account.
- Access to ServiceNow App Engine Studio and Flow Designer.
- A modern web browser. No Node.js, npm or MongoDB is needed.
- Optional: outbound email configuration, if real email delivery is wanted. Testing used the ServiceNow Outbox only.

Installation (recreating the application in a new PDI)

A new instance generates its own application scope prefix, so table names will differ from `x_2107483_automa_0_...`. Steps follow the order used in this build.

Step	Action
1	Sign in to the PDI as an administrator and open App Engine Studio.
2	Create a scoped application named Automated Network Request Management.
3	Create the two custom tables (Database Tables and Network Task Table) with the fields listed in Section 3.
4	Create the Network Request catalog item with the seven variables, choices (New, Existing; UPI, Card), defaults and required settings.
5	Create the Requester Information variable set (Opened on behalf of, Email, User Name, Phone Number) and add it to the item; configure the auto-population behaviour.
6	Create the Catalog UI Policy 'Show Existing ID for Existing Connection' (condition: type_of_connection is Existing; on load and reverse if false enabled) with an action on enter_your_existing_id that makes it visible and mandatory.
7	Create the related list on Database Tables that shows approval records, using the query on source_table and document_id.
8	Build and activate the Network Request flow (Section 3 actions), mapping each variable to the table field.
9	Build and activate the Network Task Completion Email flow.
10	Review application roles and ACLs, then run the tests in Section 10.

There are no environment variables or dependency installs; configuration is performed inside the instance.

5. Folder Structure

A ServiceNow scoped application has no client and server folders. Its components are records inside the instance, grouped below to match the Client / Server layout of the format.

Group	Components
Client side (user-facing)	Service Portal (/sp); Network Request catalog item; catalog variables; Requester Information variable set; Catalog UI Policy 'Show Existing ID for Existing Connection'.
Server side (processing)	Flow 'Network Request'; Flow 'Network Task Completion Email'; custom tables Database Tables and Network Task Table; approval related list; Email Notification 'Network Task Resolved'.
Security	Application scope, application roles, generated ACLs (Create, Read, Write, Delete).

6. Running the Application

There are no npm commands. The application runs inside the instance as soon as it is activated. To try it:

1. Open the instance's Service Portal at `/sp` and choose Network Request.

2. Fill in the form (choose New or Existing) and submit; note the request number.
3. Open the created Database Tables record and check Approval Status = Requested and the approval record.
4. Approve the request as the approver; the record should show Approved and ITIL User.
5. Create a Network Task Table record for the request manually.
6. Set the task's Work Status to Completed and check the Outbox for the 'Request has been Resolved' email.

7. API Documentation

No custom REST or SOAP endpoints were developed and no external API is integrated, so there are no request methods, parameters or JSON responses to document. The application's interfaces are internal platform triggers and actions:

Interface	Type	Purpose
Catalog Item Requested	Flow trigger	Starts the Network Request flow when a request is submitted.
Create Record (Database Tables)	Flow action	Stores the request data.
Ask For Approval / sysapproval_approver	Flow action and platform table	Creates and records the approval.
Record Updated (Network Task Table)	Flow trigger	Starts the completion email flow when Work Status changes to Completed.
Send Email	Flow action	Generates the acknowledgement and resolution emails in the Outbox.

8. Authentication

Authentication is handled by the ServiceNow platform login. The project adds no custom tokens, JWT or session code. Authorization uses the application scope, application roles and Access Control Lists:

- Application roles include an administrator role, a user role and a Database Tables user role (confirm exact scoped names in the instance).
- Generated ACLs exist for Create, Read, Write and Delete; those observed were active and use the Allow If condition type.
- Approvals are performed by the approver user (System Administrator in testing).
- This is a development-level configuration. ACLs need a least-privilege review before production. No custom encryption, firewall, identity provider or penetration testing is claimed.

9. User Interface

The screenshots below were captured from a screen recording of the running application (recorded 26 October 2026). The recipient email address in the Outbox images has been hidden.

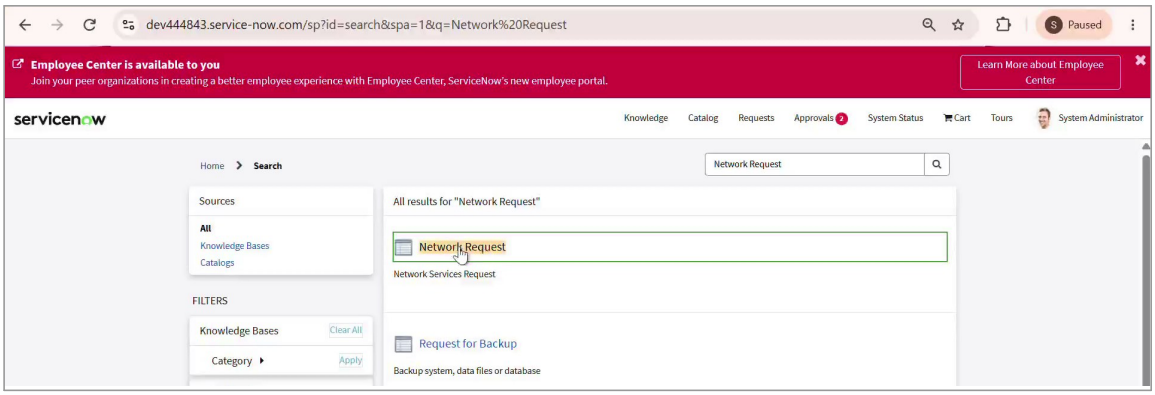


Figure 9.1: Service Portal (/sp) search results showing the Network Request catalog item.

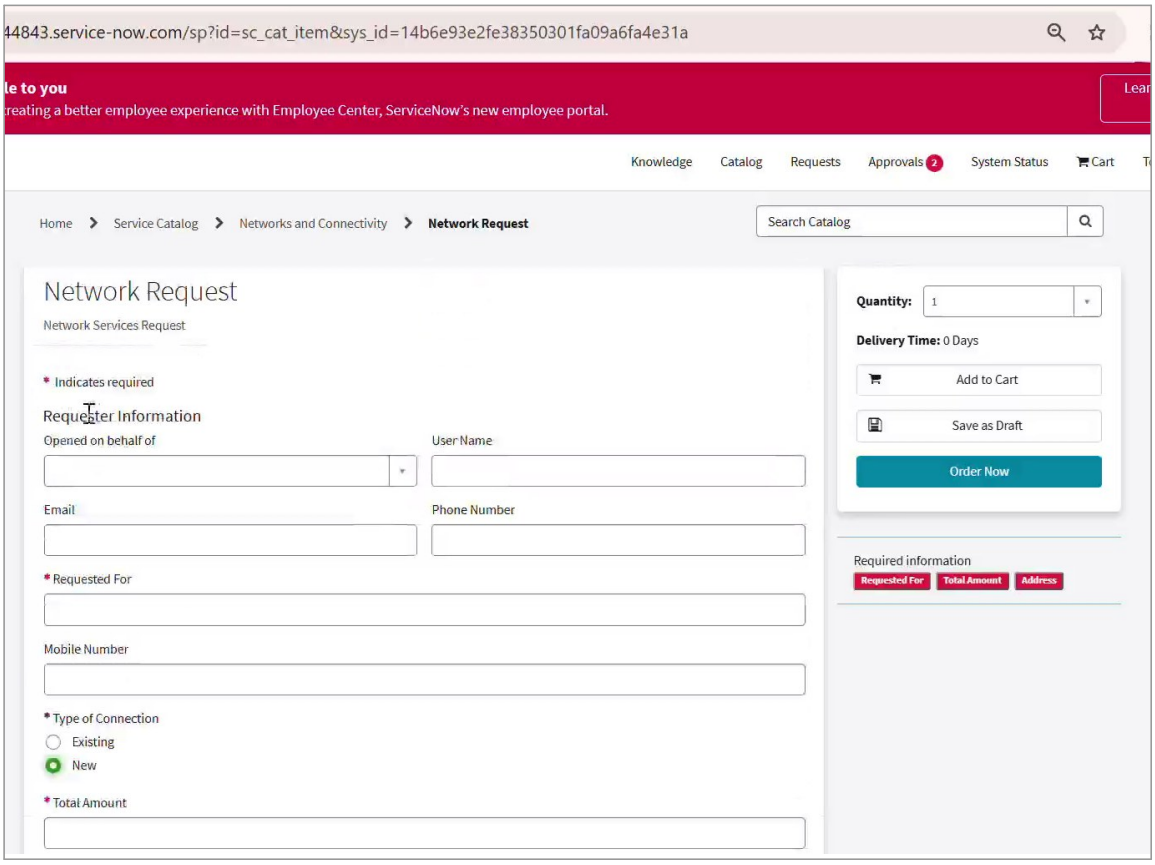


Figure 9.2: Network Request form with Type of Connection = New (default); Existing ID is hidden. Required information: Requested For, Total Amount, Address.

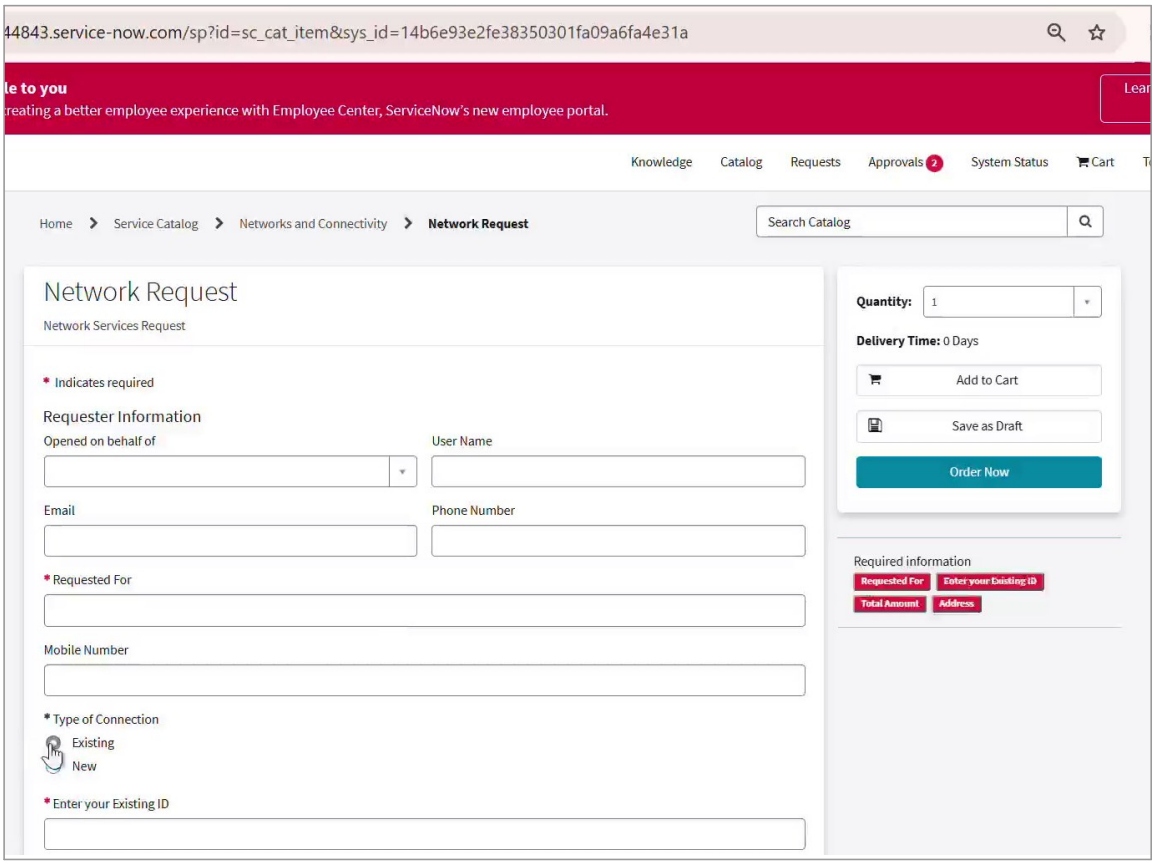


Figure 9.3: Existing selected: 'Enter your Existing ID' appears and is mandatory.

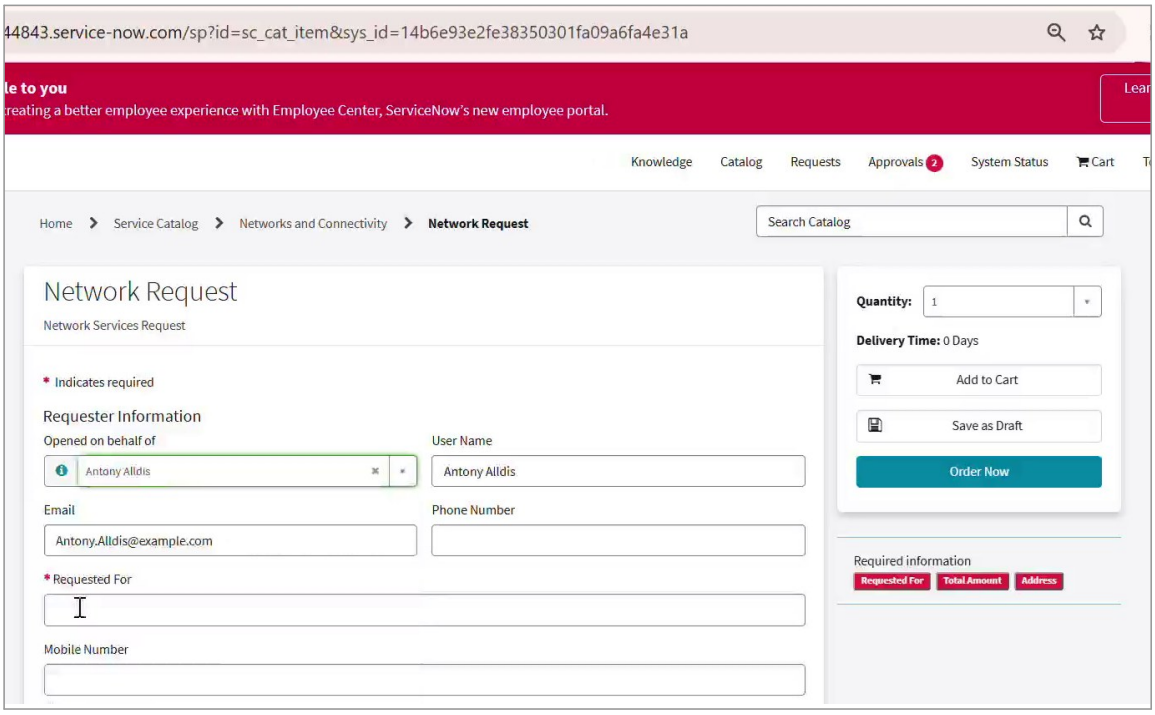


Figure 9.4: Requester Information: selecting a user fills User Name and Email. Phone Number is empty for this user.

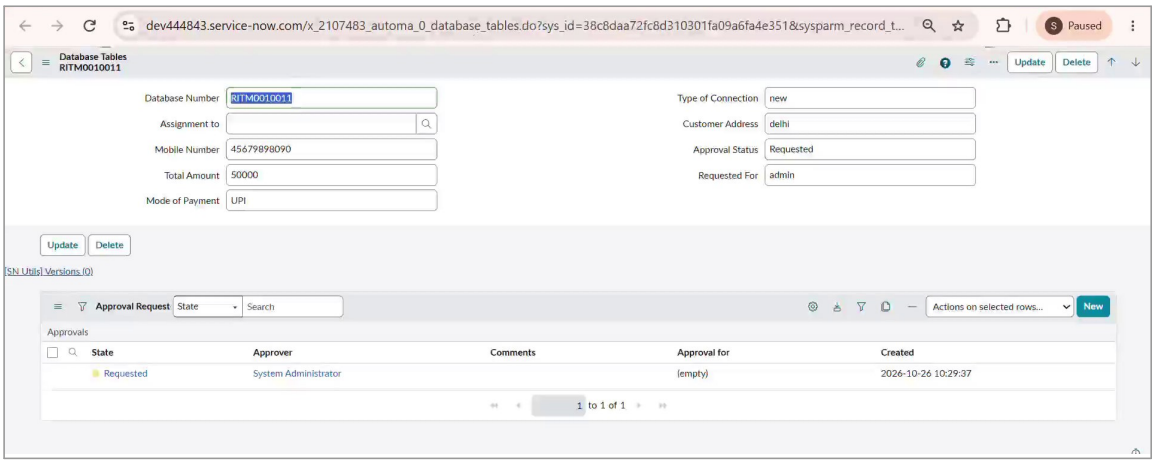


Figure 9.5: Database Tables record with Approval Status = Requested and its approval record (approver: System Administrator).

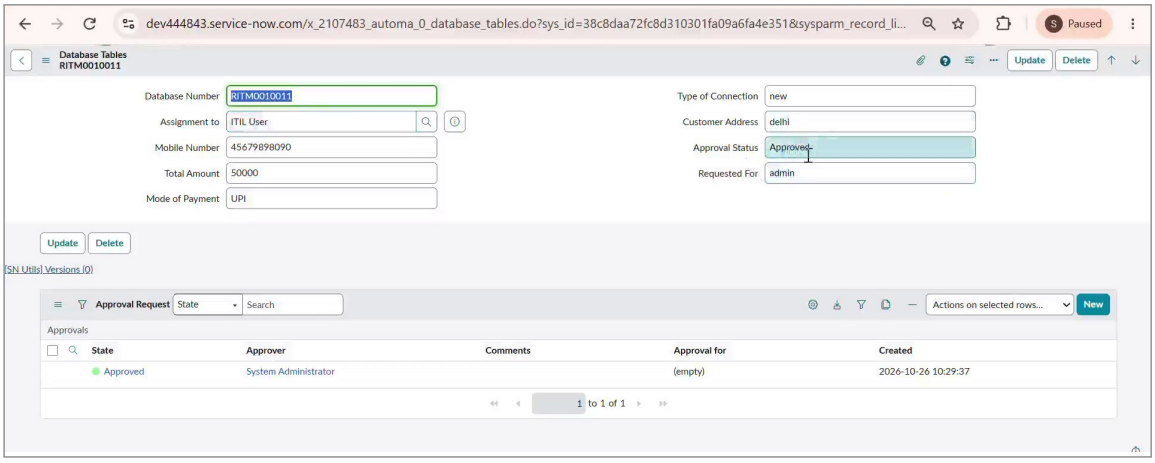


Figure 9.6: After approval: Approval Status = Approved, Assignment to = ITIL User; approval record state Approved.

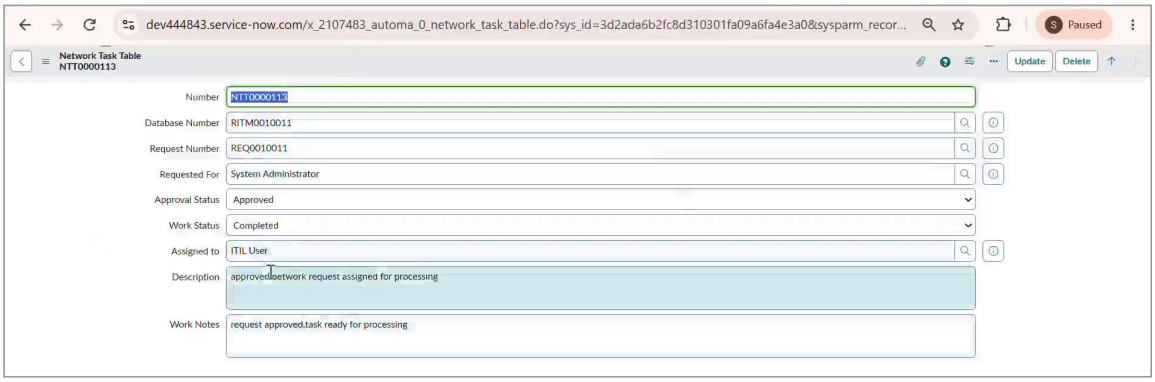


Figure 9.7: Network Task Table record with Work Status = Completed, linked Database Number and Request Number, Assigned to ITIL User.

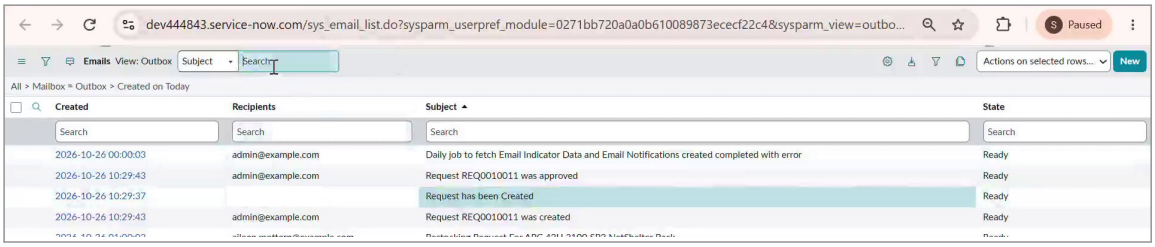


Figure 9.8: Outbox entry 'Request has been Created' (State Ready); recipient hidden.

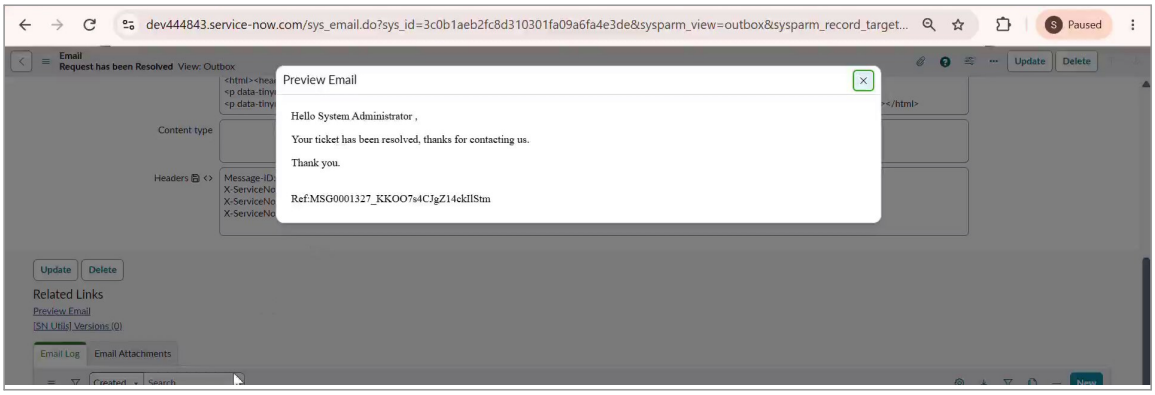


Figure 9.9: Preview of the 'Request has been Resolved' email addressed to System Administrator.

10. Testing

Testing was manual functional testing in the developer instance. No automated test tools (for example Jest, Mocha or Postman) were used. The detailed results are in the UAT report; in summary, 12 scenarios were documented: 11 observed as passing and 1 not tested (the rejection branch). The request-created email was confirmed in the Outbox (state Ready) with a preview showing the request number and the two business working day message (Figure 11.3). Test areas were request creation, conditional Existing ID, requester auto-population, the approval flow, manual task creation and the resolution email. No performance or load testing was done.

11. Screenshots or Demo

Screenshots for the user interface are in Section 9; further screenshots from the same recording follow. A screen recording of about 15 minutes covering the whole process is held by the team; no public demo link is provided. The demo order is request form, request-created email, record creation, approval, manually created task, completed task and resolution email.

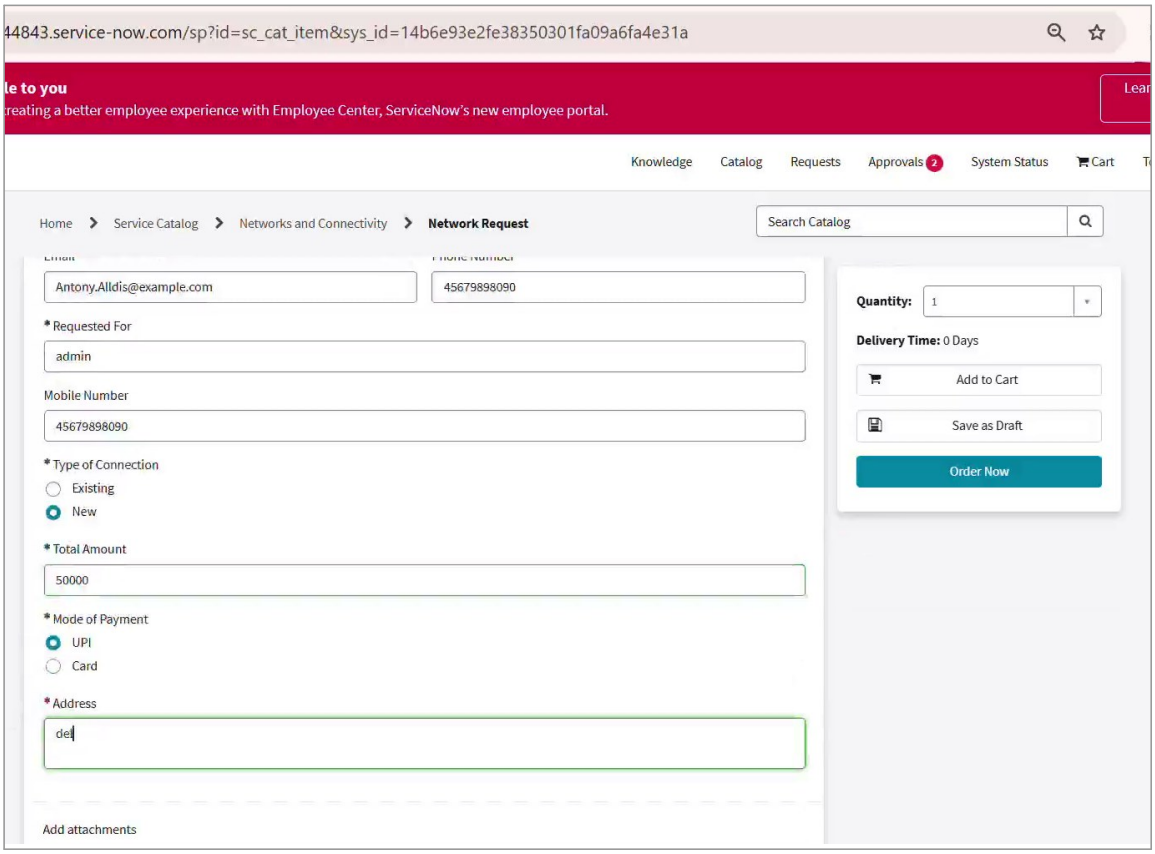


Figure 11.1: Completed request form (New connection) before ordering.

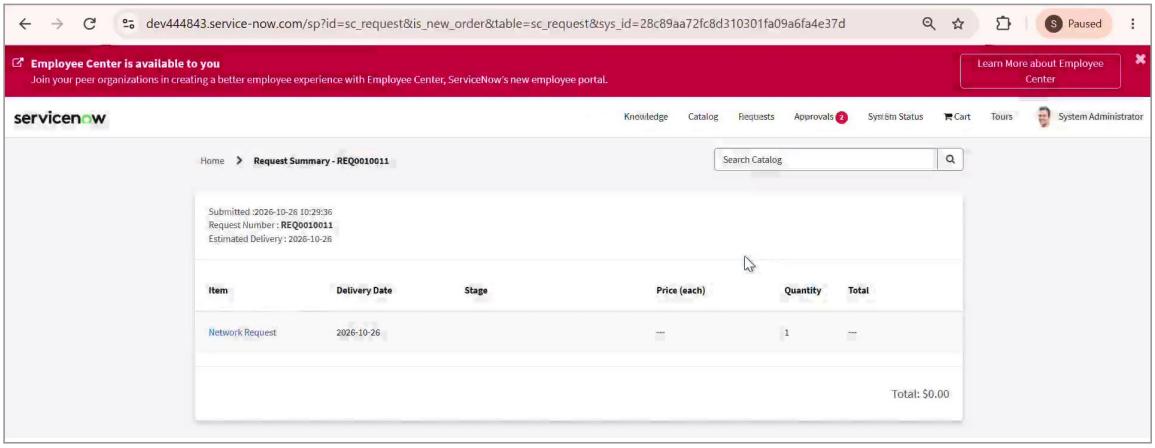


Figure 11.2: Request Summary shown after ordering (REQ0010011).

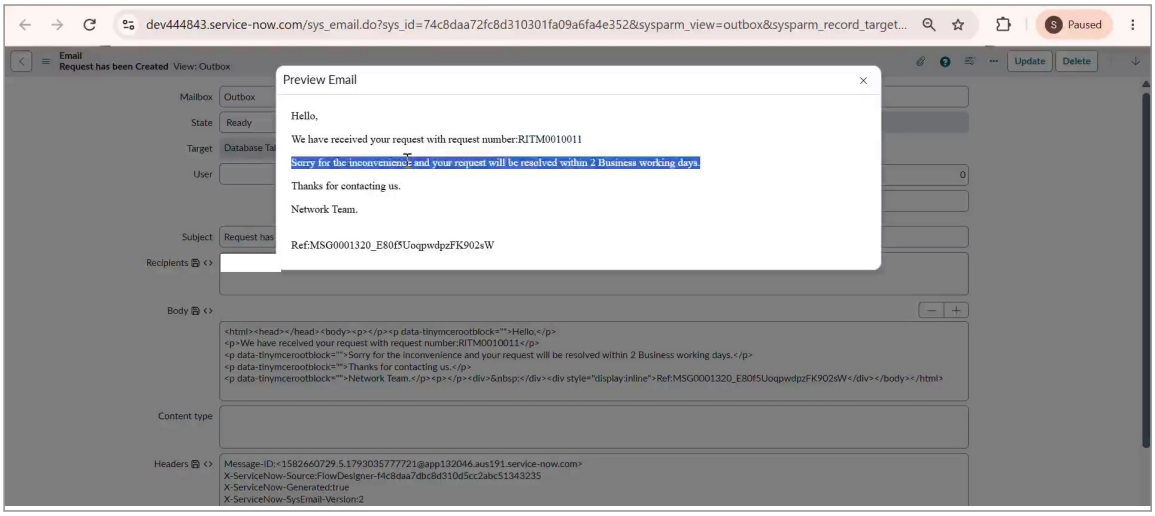


Figure 11.3: Preview of the 'Request has been Created' email: request number RITM0010011 and resolution within 2 business working days (recipient hidden).

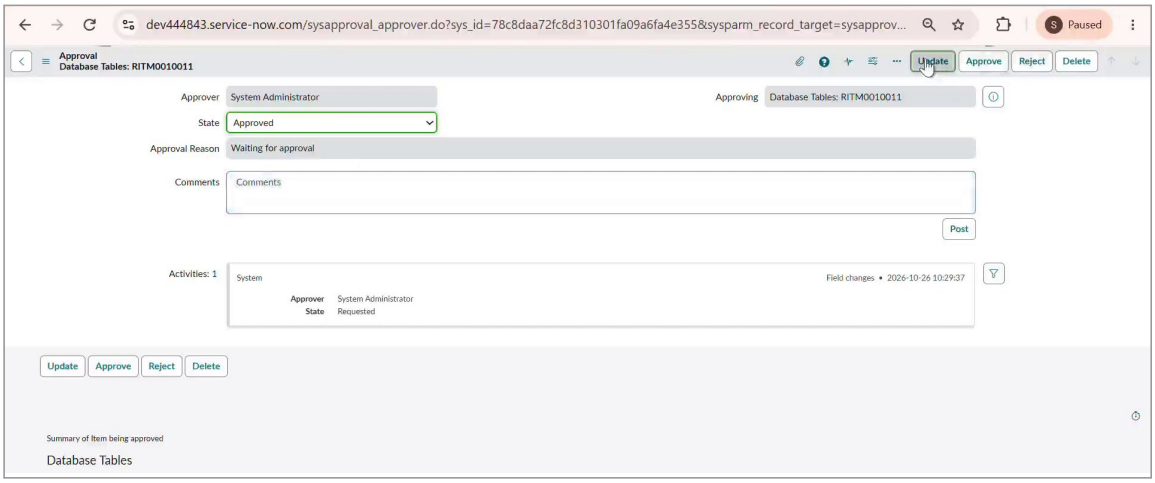


Figure 11.4: Approval record for the Database Tables record (approver: System Administrator, State: Approved).

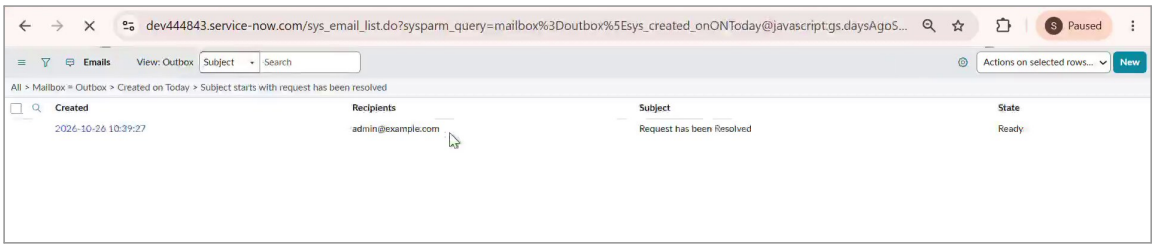


Figure 11.5: Outbox entry 'Request has been Resolved' (State Ready).

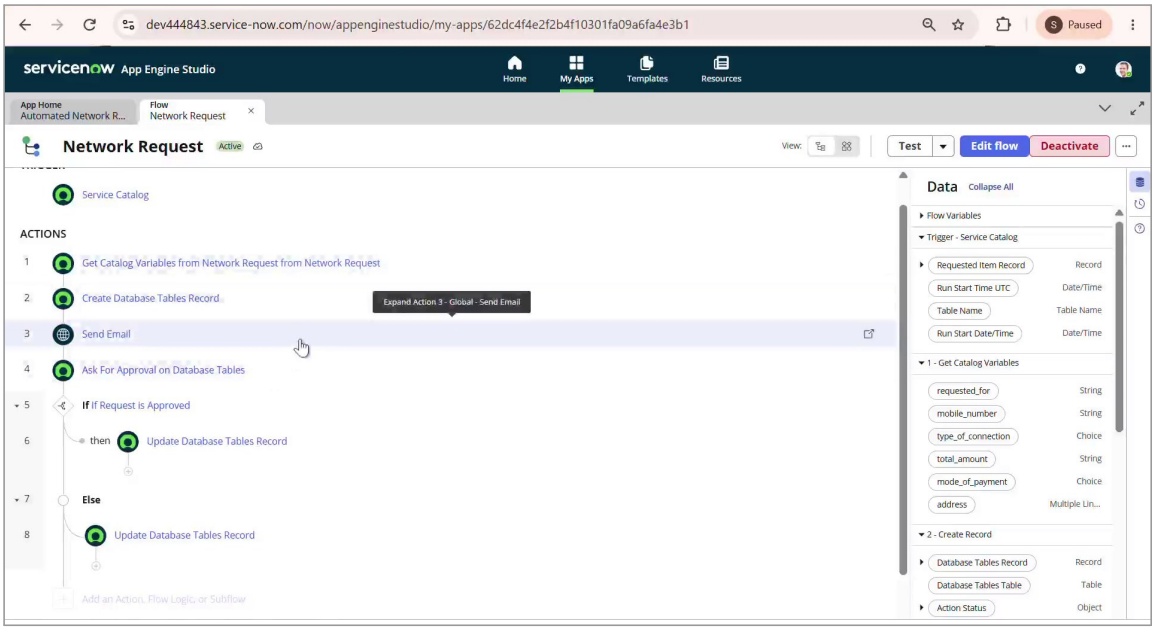


Figure 11.6: Network Request flow in Flow Designer (App Engine Studio): Get Catalog Variables, Create Record, Send Email, Ask For Approval and If / Else Update Record.

12. Known Issues

- An earlier flow execution showed missing catalog-variable warnings (address, total_amount, requested_for, mode_of_payment, mobile_number, type_of_connection). A later test stored the values, but this is not proven for all cases.
- Network Task Table records are created manually; the flow does not create them.
- The rejection branch is configured but has not been freshly tested.
- An Outbox state of Ready shows the email was queued, not that it reached an inbox.
- Mobile Number has no custom ten-digit validation script and is not marked as required on the form in the recording, although required fields were intended; Phone Number stays empty if the user's profile has no mobile phone.

- ACLs need a least-privilege review; no performance testing has been done.

13. Future Enhancements

- Create the Network Task Table record automatically after approval.
- Integrate with network provisioning tools such as Cisco DNA Center or Ansible.
- Use Integration Hub for external workflow integrations.
- Add dashboards for request volume, approval time and task completion.
- Support more request categories, such as VPN access or device provisioning.
- Introduce service-level targets and detailed audit reporting.
- Improve email delivery monitoring and failure handling.
- Add stronger field-level validation and test coverage.
- Review ACLs for least-privilege access.
- Run performance tests with realistic request volumes.
- Define deployment, update-set and rollback procedures for moving the application to another instance.